
◆ 1. Joins & Subqueries

🔍 Summary:

- **Joins** combine rows from multiple tables based on a related column.
 - **Subqueries** are nested queries used to filter or compute values inside another query.
-

✅ Example:

```
-- INNER JOIN
SELECT orders.id, customers.name
FROM orders
JOIN customers ON orders.customer_id = customers.id;

-- Subquery
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE total > 100);
```

💬 Interview Q&A:

Q1: What are the types of SQL joins? A1:

- **INNER JOIN** : common rows
 - **LEFT JOIN** : all from left + matches
 - **RIGHT JOIN** : all from right + matches
 - **FULL OUTER JOIN** : all rows + NULLs where no match
 - **CROSS JOIN** : Cartesian product
-

Q2: Difference between JOIN and SUBQUERY? A2:

- **JOIN** : Combines tables row-wise
 - **SUBQUERY** : Filters or computes values before or during the main query
-

Q3: Which is faster: JOIN or SUBQUERY? A3: JOIN is usually faster and more efficient because of better query planner optimizations.

Q4: Can a subquery be used in the SELECT clause? A4:  Yes.

```
SELECT name, (SELECT COUNT(*) FROM orders WHERE customer_id = c.id)
FROM customers c;
```

◆ 2. Indexing

Summary:

An **index** is a data structure that improves the speed of data retrieval at the cost of additional storage and slower writes.

Example:

```
CREATE INDEX idx_users_email ON users(email);
```

Interview Q&A:

Q1: What is an index in SQL? A1: A pointer structure used to speed up read operations by minimizing full table scans.

Q2: What are types of indexes? A2:

- **B-tree**: default, general use
 - **Hash**: fast equality search
 - **Full-text**: search strings
 - **Bitmap**: low-cardinality columns (rare)
-

Q3: When can indexes slow down performance? A3: During `INSERT` , `UPDATE` , `DELETE` — as indexes must be updated. Also if used unnecessarily on low-cardinality columns.

Q4: What is a covering index? A4: An index that contains all the columns needed for a query, so the database doesn't need to access the table.

◆ 3. Query Optimization

🔍 Summary:

Query optimization is the process of improving SQL query performance using indexes, reducing data scanned, and analyzing execution plans.

✅ Example:

```
-- Optimized: Only selects needed columns  
SELECT id, name FROM users WHERE active = TRUE;
```

💬 Interview Q&A:

Q1: How do you identify slow SQL queries? A1: Use `EXPLAIN` , `ANALYZE` , or the database's query execution plan tools to see cost and steps.

Q2: What are common ways to optimize queries? A2:

- Use indexes
 - Avoid `SELECT *`
 - Filter early
 - Use proper joins
 - Avoid nested loops in joins
 - Use CTEs or temp tables for large subqueries
-

Q3: What is a query plan? A3: A breakdown of how the database engine will execute a query — useful for spotting full table scans, nested loops, etc.

Q4: Does adding an index always help? A4:  No. It depends on:

- Query type
 - Data distribution
 - Whether columns are used in filters or joins
-

◆ 4. Transactions

Summary:

A **transaction** is a sequence of operations executed as a single logical unit of work, obeying the **ACID** principles (Atomicity, Consistency, Isolation, Durability).

Example:

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

Interview Q&A:

Q1: What are the ACID properties of a transaction? A1:

- **Atomicity:** All-or-nothing
 - **Consistency:** Data must remain valid
 - **Isolation:** Concurrent transactions don't interfere
 - **Durability:** Committed changes persist even on crash
-

Q2: What happens if `ROLLBACK` is issued? A2: All statements after `BEGIN` are undone and no changes are saved.

Q3: What is a dirty read? A3: Reading data that is modified by a transaction that hasn't committed yet.

Q4: How do isolation levels affect transactions? A4:

- `READ UNCOMMITTED` → allows dirty reads
 - `READ COMMITTED` → prevents dirty reads
 - `REPEATABLE READ` → prevents non-repeatable reads
 - `SERIALIZABLE` → highest isolation, most locking
-
-

◆ 5. ACID Compliance

Summary:

ACID stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability** — the set of guarantees that ensure database transactions are reliable and maintain integrity.

Interview Q&A:

Q1: What does ACID stand for? A1:

- **Atomicity:** All operations succeed or none do
 - **Consistency:** Data remains valid before and after
 - **Isolation:** Transactions do not interfere
 - **Durability:** Committed changes survive crashes
-

Q2: Which SQL command enforces Atomicity? A2: `BEGIN`, `COMMIT`, and `ROLLBACK` are used to group queries and control atomic behavior.

Q3: What happens if a crash occurs before a COMMIT ? A3: The transaction is rolled back — changes are not persisted (ensuring **Durability**).

Q4: Is every SQL database ACID-compliant? A4: Relational DBs like PostgreSQL, MySQL (InnoDB), and SQL Server are ACID-compliant; some NoSQL DBs trade off consistency for performance.

◆ 6. SQL Functions

Summary:

SQL functions are **built-in or user-defined routines** that return a single value from input values. They help transform or compute data within queries.

Example:

```
SELECT UPPER(name), ROUND(salary, 2) FROM employees;
```

Interview Q&A:

Q1: What types of SQL functions exist? A1:

- **Scalar functions:** Return a single value (UPPER , ROUND)
 - **Aggregate functions:** Operate on multiple rows (SUM , AVG , MAX)
 - **Date functions:** NOW() , DATEADD()
 - **String functions:** LEFT() , CONCAT()
-

Q2: Difference between COUNT(*) vs COUNT(column) ? A2:

- COUNT(*) : counts all rows
 - COUNT(column) : counts only non-NULL rows
-

Q3: Can you create custom SQL functions? A3:  Yes, most DBs allow **user-defined functions (UDFs)**.

Q4: What's the difference between a function and a stored procedure? A4: Functions return a value and can be used in `SELECT`. Procedures perform operations and don't return values directly.

◆ 7. Stored Procedures

Summary:

Stored procedures are saved SQL blocks that perform operations (e.g., insert, update, complex logic) and can accept parameters. Useful for automation, validation, and business logic.

Example:

```
CREATE PROCEDURE UpdateSalary(IN emp_id INT, IN bonus DECIMAL)
BEGIN
    UPDATE employees SET salary = salary + bonus WHERE id = emp_id;
END;
```

Interview Q&A:

Q1: Why use stored procedures? A1: For reusable, secure, and efficient execution of business logic close to the data.

Q2: Can stored procedures return values? A2: Yes, via **OUT parameters**, **SELECT**, or **RETURN** (depending on DB engine).

Q3: Difference between stored procedures and triggers? A3:

- **Procedure:** Manually called
 - **Trigger:** Auto-executed on data change
-

Q4: Benefits of stored procedures? A4:

- Reduce network traffic
 - Centralize logic
 - Improve security via permissions
-

◆ 8. Views

🔍 Summary:

A **view** is a virtual table defined by a SELECT query. It does **not store data**, but presents data from one or more tables in a specific structure.

✅ Example:

```
CREATE VIEW active_users AS
SELECT id, name FROM users WHERE status = 'active';
```

💬 Interview Q&A:

Q1: What is a view in SQL? A1: A saved SQL query that behaves like a table. It abstracts complexity and can be reused.

Q2: Are views up-to-date with the underlying tables? A2: ✅ Yes. Views reflect real-time changes unless they are **materialized views**.

Q3: Can you update data through a view? A3: ✅ Sometimes. Only if:

- View refers to a single table
 - Has no aggregations or joins
-

Q4: When should you use views? A4:

- To simplify complex queries
 - Abstract schema changes
 - Implement row-level security
-
-

◆ 9. Foreign Keys

Summary:

A **foreign key** creates a **relationship between two tables**, enforcing **referential integrity** by ensuring values in one table match values in another.

Example:

```
CREATE TABLE orders (  
  id INT PRIMARY KEY,  
  customer_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

Interview Q&A:

Q1: What is a foreign key in SQL? A1: A column or set of columns that references a primary key in another table to enforce data consistency.

Q2: What happens when referenced data is deleted? A2: Depends on the **ON DELETE** rule:

- **CASCADE** : deletes child rows
 - **SET NULL** : sets FK to NULL
 - **RESTRICT** / **NO ACTION** : prevents deletion
-

Q3: Can a table have multiple foreign keys? A3:  Yes. A table can reference multiple tables via separate foreign keys.

Q4: What is the difference between primary and foreign keys? A4:

- **Primary key:** Unique identifier in its own table
- **Foreign key:** References a primary key in another table

◆ 10. SQL Triggers

Summary:

A **trigger** is a piece of logic that **automatically executes** when an event (INSERT, UPDATE, DELETE) happens on a table.

Example:

```
CREATE TRIGGER log_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
INSERT INTO salary_audit(emp_id, old_salary, new_salary)
VALUES (OLD.id, OLD.salary, NEW.salary);
```

Interview Q&A:

Q1: What is a trigger? A1: A database event handler that runs automatically when certain changes happen to a table.

Q2: Types of triggers? A2:

- BEFORE INSERT / UPDATE / DELETE
- AFTER INSERT / UPDATE / DELETE
- Some DBs support INSTEAD OF triggers (e.g., on views)

Q3: Use cases for triggers? A3:

- Auditing changes
 - Enforcing business rules
 - Auto-updating log tables
-

Q4: Can triggers affect performance? A4:  Yes. Poorly written triggers can cause slowdowns or recursion if not handled properly.

◆ 11. Views vs Tables

Summary:

A **table** stores persistent data, while a **view** is a virtual table created by a query. Views abstract complexity but don't store data.

Interview Q&A:

Q1: What is the main difference between a view and a table? A1:

- **Table:** Stores actual data
 - **View:** Stores a SQL definition; generates result at runtime
-

Q2: Can views have indexes? A2:  No, unless you're using **materialized views** (which persist data and may be indexed).

Q3: When should you use views instead of tables? A3:

- To simplify complex joins
 - Abstract business logic
 - Restrict column/row access
-

Q4: Are views automatically updated? A4:  Yes. Standard views reflect the latest data each time they are queried.

◆ 12. Permissions

Summary:

Permissions (aka privileges) define **who can do what** (SELECT, INSERT, UPDATE, DELETE) on tables, views, and other objects. Often managed using **GRANT** and **REVOKE**.

Example:

```
GRANT SELECT, INSERT ON employees TO analyst_user;  
REVOKE DELETE ON employees FROM analyst_user;
```

Interview Q&A:

Q1: What is the difference between **GRANT** and **REVOKE** ? A1:

- **GRANT** : gives access
 - **REVOKE** : removes access
-

Q2: Can you restrict permissions to specific columns? A2:  Yes. Some databases support **column-level** privileges.

Q3: What is **role-based access control (RBAC)**? A3: Assigning permissions to **roles**, and then assigning users to those roles, rather than managing individual permissions.

Q4: Why is permission management important? A4: For **data security**, **regulatory compliance**, and preventing accidental data loss.

◆ 13. Query Plans

Summary:

A **Query Execution Plan** (or **Explain Plan**) shows how the SQL engine will execute a query — including steps like scans, joins, index usage, and estimated cost.

Used to **debug slow queries** and optimize performance.

Example:

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;
```

Interview Q&A:

Q1: What is a query execution plan? A1: It is a visual/textual breakdown of how a query is executed — including operations like index scans, joins, sort, filter, etc.

Q2: What is the difference between full table scan and index scan? A2:

- **Full table scan:** Reads every row
 - **Index scan:** Uses an index to locate relevant rows faster
-

Q3: How do you interpret query cost in a plan? A3: Lower cost = faster expected performance (approx). Cost depends on CPU, I/O, memory access, row count.

Q4: How can query plans help in optimization? A4: They show performance bottlenecks (e.g., missing indexes, wrong join order, sort operations).

14. Stored Views

Summary:

A **stored view** is a **predefined virtual table** created from a SELECT query. It allows reusability and abstraction but does not store data unless materialized.

✓ Example:

```
CREATE VIEW monthly_sales AS
SELECT product_id, SUM(amount) AS total FROM sales
GROUP BY product_id;
```

💬 Interview Q&A:

Q1: Are views physically stored in the DB? A1: No — they're executed on demand unless they are **materialized views**.

Q2: What are the pros of stored views? A2:

- Simplify complex queries
 - Reuse logic
 - Improve security (expose limited data)
-

Q3: Can you join views in queries? A3: ✓ Yes. Views can be treated like tables in queries.

Q4: Can stored views degrade performance? A4: ✓ Yes, especially if they include joins or aggregations — since views are re-evaluated each time they're queried.

◆ 15. Views & CTEs (Common Table Expressions)

🔍 Summary:

CTEs are temporary, named result sets defined inside a query using the `WITH` clause. Used for **readability**, **recursion**, and breaking down complex logic.

✓ Example:

```
WITH recent_orders AS (  
    SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '7 days'  
)  
SELECT customer_id, COUNT(*) FROM recent_orders GROUP BY customer_id;
```

💬 Interview Q&A:

Q1: Difference between a CTE and a view? A1:

- **CTE:** Temporary, used within a query
- **View:** Persistent, reusable across queries

Q2: Can you nest multiple CTEs? A2: ✓ Yes. You can define multiple CTEs before the main query.

Q3: Use cases for CTEs? A3:

- Simplify complex joins or subqueries
- Recursive queries (e.g., hierarchical data)
- Break down steps for clarity

Q4: Are CTEs faster than subqueries? A4: Not always. CTEs improve **readability** and **debuggability**, but performance depends on query planner and reuse.

◆ 16. Query Execution

🔍 Summary:

Query Execution refers to how SQL statements are parsed, optimized, and run by the DB engine — involving multiple steps from parsing to returning results.

Interview Q&A:

Q1: What are the stages of query execution? A1:

1. **Parsing:** Syntax check
 2. **Binding:** Resolve tables/columns
 3. **Optimization:** Choose best execution path
 4. **Execution:** Run and return results
-

Q2: What impacts query execution time? A2:

- Index usage
 - Joins & sort steps
 - Disk I/O
 - Memory availability
 - Query logic (e.g., using `LIKE '%abc'`)
-

Q3: What is a “sargable” query? A3: A query that can use an index efficiently. E.g., `WHERE name LIKE 'abc%'` is sargable; `LIKE '%abc'` is not.

Q4: How to reduce execution time of complex queries? A4:

- Add indexes
 - Use proper joins
 - Avoid redundant subqueries
 - Reduce result set size
 - Use `EXISTS` or `IN` properly
-
-

◆ 17. Constraints

 Summary:

Constraints are rules applied to table columns to enforce **data integrity**, such as unique values, required fields, value ranges, or foreign key relationships.

✓ Example:

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  email VARCHAR(255) UNIQUE,  
  age INT CHECK (age >= 18),  
  country_id INT NOT NULL  
);
```

💬 Interview Q&A:

Q1: What are common SQL constraints? A1:

- **PRIMARY KEY** : Unique + NOT NULL
 - **UNIQUE** : No duplicate values
 - **NOT NULL** : Must provide value
 - **CHECK** : Enforce logical rules
 - **FOREIGN KEY** : Enforce relational integrity
-

Q2: What is the difference between **UNIQUE and **PRIMARY KEY** ? A2:** Both enforce uniqueness, but:

- **PRIMARY KEY** = unique + not null
 - A table can only have **one primary key** but **multiple unique keys**
-

Q3: What does the **CHECK constraint do? A3:** Validates values against a logical condition (e.g., **salary > 0**).

Q4: Can constraints be deferred until the end of a transaction? A4: In some DBs (like PostgreSQL), yes — using **DEFERRABLE INITIALLY DEFERRED**.

◆ 18. Data Consistency

Summary:

Data consistency ensures that all database rules (constraints, relationships, and transactions) maintain valid and accurate data at all times.

Interview Q&A:

Q1: How is data consistency maintained in SQL databases? A1: Through:

- ACID transactions
 - Constraints
 - Referential integrity (foreign keys)
-

Q2: What could lead to inconsistent data? A2:

- Skipping foreign keys
 - Poor error handling in transactions
 - Concurrent writes without isolation
 - Manual updates outside app logic
-

Q3: Can consistency be sacrificed for performance? A3: Yes — in some **NoSQL** systems (eventual consistency). But in RDBMS, consistency is a primary design feature.

Q4: How do constraints relate to consistency? A4: Constraints like `NOT NULL`, `UNIQUE`, and `FOREIGN KEY` enforce consistent and valid data entry.

◆ 19. SQL Security

Summary:

SQL Security involves controlling **who can access what**, and how data is protected from unauthorized access or accidental modifications.

Interview Q&A:

Q1: What is the principle of least privilege in SQL? A1: Grant only the **minimum permissions** needed for a user to do their job — e.g., only `SELECT` on a read-only reporting user.

Q2: What types of access control exist? A2:

- Discretionary Access Control (DAC): Permissions assigned to users
 - Role-Based Access Control (RBAC): Roles granted to users
 - Column-level and row-level security
-

Q3: What is SQL injection and how do you prevent it? A3: A vulnerability where attackers insert malicious SQL via input. Prevent using **parameterized queries**, **ORMs**, and **input validation**.

Q4: Can we encrypt data in SQL databases? A4:  Yes. You can encrypt:

- At rest (disk-level)
 - In transit (SSL/TLS)
 - Column-level (Transparent Data Encryption / custom logic)
-

20. Replication

Summary:

Replication is the process of copying data from one database server to another, often used for **high availability**, **failover**, or **scaling reads**.

Interview Q&A:

Q1: What are the types of replication? A1:

- **Master-slave (primary-replica):** Writes go to master, reads from replicas
 - **Multi-master:** Multiple nodes can write (conflict risk)
 - **Snapshot, log-based, or statement-based** replication
-

Q2: How does replication help with performance? A2: By allowing **read-scaling** — queries are offloaded to replicas, reducing load on the primary DB.

Q3: What are replication challenges? A3:

- Lag (replica delay)
 - Conflicts in multi-master setup
 - Ensuring data consistency and failover management
-

Q4: How is replication different from backup? A4:

- **Replication:** Real-time or near real-time sync
 - **Backup:** Snapshot of the DB at a point in time (used for recovery)
-
-

◆ 21. Partitioning

Summary:

Partitioning breaks a large table into **smaller, manageable pieces** called **partitions** for better performance and easier maintenance.

✓ Example (PostgreSQL):

```
CREATE TABLE orders (  
  id INT,  
  order_date DATE,
```

```
amount DECIMAL
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024 PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

Interview Q&A:

Q1: Why use table partitioning? A1:

- Speed up queries by scanning only relevant partitions
 - Improve load time
 - Easier archiving and purging
-

Q2: Types of partitioning? A2:

- Range (e.g., by date)
 - List (e.g., by region)
 - Hash (e.g., evenly distribute IDs)
 - Composite (mix of above)
-

Q3: Difference between sharding and partitioning? A3:

- **Partitioning:** Done within a single DB
 - **Sharding:** Across multiple DB instances
-

Q4: Can indexes be used on partitions? A4:  Yes, indexes can be applied to individual partitions or globally (depends on DB engine).

22. Window Functions

Summary:

Window functions perform calculations across a set of rows **related to the current row**, without collapsing them (unlike GROUP BY).

✓ Example:

```
SELECT
  employee_id,
  department,
  salary,
  RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept
FROM employees;
```

💬 Interview Q&A:

Q1: Key difference between **GROUP BY** and window functions? A1:

- **GROUP BY** : Aggregates rows into one
 - **WINDOW** : Keeps all rows, adds computed values
-

Q2: Common window functions? A2:

- **ROW_NUMBER()**
 - **RANK()**
 - **DENSE_RANK()**
 - **LEAD()** / **LAG()**
 - **SUM() OVER (...)**
-

Q3: What does **PARTITION BY** do in window functions? A3: Divides the result set into groups for applying the function.

Q4: Use cases for window functions? A4:

- Running totals
- Leaderboards
- Time-based analysis

- Row comparisons (e.g., current vs previous)

◆ 23. Materialized Views

Summary:

A **materialized view** stores the result of a query physically on disk — useful for expensive queries that don't need real-time freshness.

✓ Example (PostgreSQL):

```
CREATE MATERIALIZED VIEW sales_summary AS
SELECT region, SUM(amount) AS total
FROM sales
GROUP BY region;

-- To refresh:
REFRESH MATERIALIZED VIEW sales_summary;
```

Interview Q&A:

Q1: Difference between normal view and materialized view? A1:

- **View:** Executes query live every time
- **Materialized View:** Stores results; needs refresh

Q2: Use cases for materialized views? A2:

- Heavy aggregations
- Dashboards
- Reporting on slow queries

Q3: Can you index materialized views? A3:  Yes, since they're stored like a real table.

Q4: Does data in materialized views auto-update? A4: ❌ No. You must manually refresh it (or schedule it).

◆ 24. Normalization & Denormalization

🔍 Summary:

- **Normalization:** Organizes data into smaller, related tables to **reduce redundancy** and ensure integrity.
 - **Denormalization:** Combines data to reduce joins and **improve read performance**.
-

✅ Normalization Example:

Split customer name and address into separate tables to avoid repeating them in orders.

💬 Interview Q&A:

Q1: What are the normal forms in SQL? A1:

- 1NF: Atomic columns
 - 2NF: No partial dependency on PK
 - 3NF: No transitive dependencies
-

Q2: When to denormalize data? A2:

- For analytics/reporting
 - When performance matters more than storage
 - In star schema/data warehouse designs
-

Q3: Risks of denormalization? A3:

- Data inconsistency
- Redundant storage

- Complicated updates

Q4: Is normalization always preferred? A4: Mostly for OLTP systems; for OLAP/data warehouses, denormalization is common.

◆ 25. SQL Pitfalls & Anti-patterns

Summary:

Common mistakes that lead to **poor performance**, **bugs**, or **scalability issues** in SQL.

! Common Pitfalls:

Problem	Fix/Tip
<code>SELECT *</code>	Always select only needed cols
No indexes on joins	Index foreign/join keys
<code>LIKE '%abc'</code>	Use prefix search (sargable)
Too many joins	Break into subqueries or CTEs
Implicit type casting	Use correct column data types

Interview Q&A:

Q1: Why is `SELECT *` discouraged? A1: Fetches unnecessary data, breaks if schema changes, slows performance.

Q2: What is a non-sargable query? A2: One that prevents index use. E.g., `WHERE LOWER(name) = 'john'` blocks index; better to store normalized data.

Q3: Why is implicit type conversion dangerous? A3: Can result in full scans and bugs (e.g., comparing a string to an int forces conversion and breaks performance).

Q4: How to avoid anti-patterns? A4:

- Use EXPLAIN
 - Follow naming conventions
 - Use indexes wisely
 - Test with large datasets
-